Laboratório Funcional · Aula 12

Maria Eduarda de Castro Lachimia · RA 24055202-2 · Turma A
Gerado em 16/10/2026, 21:44:15 · tempo registrado no cronômetro: 45:19

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

def sorteia(): return random.randint(1, 6): gera um número aleatório, portanto, o resultado pode variar entre chamadas

def poe(lista, x): lista.append(x); return lista: modifica a lista original usando append()

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

1 = A-1 A-2

4 = 14

7 = [7, 3, 9] [3, 7, 9]

Saídas (original e com compor(g, f))

```
original:
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

```
Com compor(g,f):
1: A-1 A-2
2: True
3: 8
4: 18
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

A função proximo_id("A") devolve valores diferentes porque depende de uma variável global chamada contador, que é modificada a cada chamada. Isso dificulta os testes porque o resultado depende do estado anterior do programa, e não apenas do argumento fornecido. Assim, chamar a mesma função com a mesma entrada pode gerar resultados diferentes

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

Na função compor(f, g)(4), o programa executa primeiro a função g, que multiplica o valor por 3, e depois a função f, que soma 2 ao resultado. Assim, g(4) = 3 × 4 = 12 e, em seguida, f(12) = 12 + 2 = 14. Portanto, o resultado é 14. Já na função compor(g, f)(4), a ordem é invertida: primeiro, a função f soma 2 ao valor, resultando em f(4) = 4 + 2 = 6; depois, a função g multiplica esse resultado por 3, ou seja, g(6) = 3 × 6 = 18. Portanto, o resultado é 18. A diferença acontece porque a ordem em que as funções são executadas altera o resultado final: primeiro somar 2 e depois multiplicar por 3 não produz o mesmo resultado que primeiro multiplicar por 3 e depois somar 2.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

Os dois comandos ordenam os elementos de uma lista, mas funcionam de maneiras diferentes, sorted(notas) Cria uma nova lista ordenada, sem modificar a lista original e notas.sort() Ordena a própria lista, modificando-a diretamente.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
original:

22
14
25
8
negativo
A
(B C)
(A B C)
```

```
((A B) C D)
B

sem apóstrofo:

22
14
25
8
negativo
Backtrace:
           5 (primitive-load "/box/main.scm")
In ice-9/eval.scm:
   191:35  4 (_ #f)
   191:35  3 (_ #f)
   196:27  2 (_ #f)
   223:20  1 (proc #<directory (guile-user) 13477426cc80>)
In unknown file:
           0 (%resolve-variable (7 . A) #<directory (guile-user) 134…>)

ERROR: In procedure %resolve-variable:
Unbound variable: A
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo em Scheme serve para indicar que uma expressão deve ser tratada como dado, sem ser executada. Sem o apóstrofo, o Scheme tentou interpretar (A B C) como uma chamada de função, considerando A como o nome de uma função.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C))))

saída:

22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

em Scheme, provavelmente vai funcionar. Em Python, a mesma função provavelmente dará erro

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
[Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, cresce a pilha de chamadas, porque cada chamada precisa esperar a próxima terminar para realizar a operação que falta. Já na recursão em cauda, a chamada recursiva é a última operação da função, pois o cálculo é feito antes e o resultado parcial é passado como argumento.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme conseguiu fazer um milhão de chamadas porque possui otimização de chamada em cauda, que permite reutilizar o espaço da pilha quando a chamada recursiva é a última operação da função. Já o Python não possui essa otimização e acumula chamadas na pilha até atingir o limite máximo de recursão, gerando o erro.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))));


(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))));


(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis))
         (cons (car lis) (pares (cdr lis))))
```

```
        (else
         (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi entender como a recursão percorre a lista até ela ficar vazia e como combinar car e cdr para processar cada elemento.
Usei null? para verificar se a lista está vazia, car para acessar o primeiro elemento e cdr para obter o restante da lista. Na função tamanho, usei null? como caso de parada e cdr para continuar contando os elementos. Na função soma-quadrados, usei null? para encerrar a soma, car para pegar o número e calcular seu quadrado, e cdr para continuar com os demais. Na função pares, usei null? para encerrar a recursão, car junto com even? para verificar se o primeiro número é par, cons para adicioná-lo à lista de resultados e cdr para avançar para o próximo elemento.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

[0,1,4,9,16]

Saídas (original e com member 17)

```
original:

2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42

com member 17:
Time limit exceeded
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell consegue imprimir parte de uma lista infinita por causa da avaliação preguiçosa (lazy evaluation). Ele só calcula os elementos necessários para produzir o resultado.
A expressão primeiro 42 (div 1 0) não dá erro porque a função primeiro x _ = x retorna o primeiro argumento e ignora o segundo. Como o Haskell só avalia uma expressão quando seu valor é necessário, div 1 0 nunca é executado e a função retorna 42.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

A função member 17 squares nunca termina porque squares é uma lista infinita e os quadrados são 0, 1, 4, 9, 16, 25, 36, ... Como 17 não é um quadrado perfeito, a função continua procurando indefinidamente, sem encontrar o valor e sem saber quando parar. Podemos aproveitar o fato de que a lista está em ordem crescente e interromper a busca quando o elemento atual for maior que o número procurado.

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5): 15 e [1, 2, 3].map(vezes(5)): [5, 10, 15]

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

Isso acontece por causa do fechamento (closure). Quando somador(10) é executada, ela devolve uma função que mantém acesso à variável n, cujo valor é 10. Mesmo depois que somador termina de executar, a função devolvida continua tendo acesso a esse valor. Assim, quando mais10(5) é chamada, ela calcula 5 + 10, retornando 15.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Os resultados são diferentes porque as funções são executadas em ordens diferentes. Em f.compose(g).apply(4), a função g é executada primeiro: g(4) = 3 × 4 = 12. Depois, f(12) = 12 + 2 = 14. Já em f.andThen(g).apply(4), a função f é executada primeiro: f(4) = 4 + 2 = 6. Depois, g(6) = 3 × 6 = 18. Portanto, compose retorna 14 e andThen retorna 18. O método f.compose(g) corresponde a compor(f, g) da Prática 1, pois ambos executam primeiro g e depois f.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);
        int soma = 0;
        for (int n : nums) {
            if (n % 2 == 0) {
                soma += n * n;
            }
        }
        System.out.println(soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
```

```
  { nome: "Davi", nota: 9.5 },
];
const aprovados = [];
for (let i = 0; i < alunos.length; i++) {
  if (alunos[i].nota >= 6) {
    aprovados.push(alunos[i].nome.toUpperCase());
  }
}
console.log(aprovados.join(", "));
```

Saídas das suas versões

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);

        int soma = nums.stream()
            .filter(n -> n % 2 == 0)
            .mapToInt(n -> n * n)
            .sum();

        System.out.println(soma);
    }
}
```

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];

const aprovados = alunos
  .filter(aluno => aluno.nota >= 6)
  .map(aluno => aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional costuma ser mais fácil de ler porque é mais curta e deixa claro o objetivo do programa, como filtrar alunos aprovados ou somar os quadrados dos números pares. Já a versão com laço pode ser mais fácil de depurar, pois permite acompanhar cada iteração e verificar os valores intermediários com mais facilidade. Na versão funcional, também é possível depurar, mas pode ser necessário separar as etapas do pipeline para inspecionar os resultados. Portanto, a melhor escolha depende do objetivo: a funcional favorece a clareza e a concisão; a versão com laço pode facilitar o acompanhamento passo a passo durante a depuração.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão funcional em Haskell é mais fácil de entender porque é mais curta e expressa diretamente a operação: calcular os cubos dos números de 1 até n e somá-los. Eu aceitaria o custo de execução em sistemas pequenos, protótipos ou aplicações em que a clareza do código é mais importante que o desempenho.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi perceber que é possível resolver problemas sem laços explícitos e que a avaliação preguiçosa permite trabalhar até com listas infinitas.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

não respondido